

Testing — In Plain English

What we test in the Time-Off service, why it matters, and proof that it all passes. Written to be readable without any technical background.

The short version

- **67 automated checks. Every one passes.**
 - They cover about **92% of the code** — and the most important logic (the day-counting) is covered **100%**.
 - Anyone can run them with one command: `npm test`.
-

Two kinds of tests

1. **Small focused checks (42 of them).** Each one tests a single piece of logic on its own — for example, "does the day-counting add up?" or "does a retry eventually give up and raise an alert?"
2. **Whole-service checks (25 of them).** These start the **entire service**, plus a realistic stand-in for the HCM, and use it exactly like a real employee or manager would — by sending real web requests.

To make the whole-service checks realistic, we built a **pretend HCM** that behaves like the real one: it can be slow, it can refuse things, and it can change balances on its own (a bonus, a yearly reset). That lets us rehearse real-world situations safely.

What we make sure works

Each line below is something we check automatically. A ✓ means there is a test proving it.

Requesting time off

- ✓ You get an **instant** yes-or-no — no waiting on the HCM.
- ✓ You **can't ask for more days than you have**. And even if the HCM were misconfigured to allow it, we still block the over-spend on our side first.
- ✓ Asking for an **unknown** person or location is refused with a clear message.
- ✓ **Bad input** is refused clearly — for example a wrongly-formatted date, or an end date that comes before the start date.
- ✓ Sending the **same request twice by accident** only counts once.

Approving, rejecting, and cancelling

- ✓ **Approving** sets the days aside and then records them with the HCM automatically.
- ✓ **Rejecting** gives the days straight back and never touches the HCM.
- ✓ **Cancelling** gives the days back. If they had already been recorded with the HCM, we automatically file a **refund**.
- ✓ You **can't approve the same request twice**, and you can't cancel one that's already finished. Acting on a request that doesn't exist returns a clear "not found."

Keeping balances correct

- ✓ **Available days = total days – days on hold**. This stays correct across a whole mix of actions (several requests, an approval, and a deduction).

- ✓ When **many requests arrive at the same instant**, we never give away more than exists. *Tested example*: 10 requests against a balance of 5 → exactly **5 succeed, 5 are refused**, and the balance never goes negative.
- ✓ Every change is written to a **history log**, and we can rebuild the current balance just by replaying that log — proof the numbers always add up.

Staying in step with the HCM

- ✓ A **bonus** added at the HCM (e.g. a work anniversary) shows up for us after a sync.
- ✓ A **start-of-year reset** is picked up for everyone.
- ✓ The HCM can **push** an update to us, and we apply it.
- ✓ An **old update that arrives late is ignored**, so it can't undo newer information.
- ✓ We **never accidentally refund** days because of timing: if the HCM's snapshot is slightly out of date and misses a deduction we just made, we keep the correct (lower) balance.
- ✓ If the HCM ever shows **fewer days than people have already reserved**, we **flag it** for review instead of silently breaking anything.

When the HCM misbehaves

- ✓ **HCM temporarily down** → we keep retrying until it works, and your hold stays safe in the meantime.
- ✓ **HCM stays down** → after several failed tries we **park** the task and keep the hold, so nothing is lost and a person can step in.
- ✓ **HCM firmly refuses** → we return the days, mark the request failed, and re-check the real balance.

Proof it all passes

This is the actual result of running the full test suite (`npm run test:cov`):

```
Test Suites: 11 passed, 11 total
Tests:      67 passed, 67 total
Time:       about 7 seconds
```

How much of the code is exercised by the tests:

Measure	Covered	What it means
Lines of code	94.7%	Nearly every line runs during testing
Statements	92.5%	Almost all individual actions are tested
Functions	93.8%	Almost every function is called by a test
Decision branches	72.1%	Most yes/no paths are tested (the rest are mostly defensive "just in case" code)

The core day-counting logic — the part where a mistake would cost someone real vacation days — is covered **100%**.

Want to see the detail?

Every plain statement above is backed by one or more named tests in the code:

- Small focused checks live next to the code, in files ending `.spec.ts`.
- Whole-service checks live in the `test/e2e/` folder, in files ending `.e2e-spec.ts`.

Run `npm run test:cov` to run everything and open the full, line-by-line coverage report it generates under the `coverage/` folder.